

第五章 程序的执行

程序执行和指令执行概述

流水线方式下指令的执行

程序的执行机制

- **主要教学目标**
 - **理解CPU如何控制程序的执行流**
 - **了解一条指令的执行过程**
 - **了解CPU的主要功能和内部结构**
 - **了解数据通路的基本组成和定时方式**
 - **理解指令执行时数据通路中信息的流动过程**
 - **了解指令流水线的基本概念**
 - **了解内部异常和外部中断的基本概念**

详细内容参看《计算机组成与系统结构（第2版）》（袁春风等，清华大学出版社）

程序的执行机制

◦ 分以下三个部分介绍

• 第一讲：程序执行概述

- 程序及指令的执行过程
- CPU的基本功能和基本组成

• 第二讲：流水线方式下指令的执行

- 指令流水线的基本原理
- 适合流水线的指令集特征
- CISC和RISC风格指令集
- 指令流水线的实现
- 高级流水线实现技术

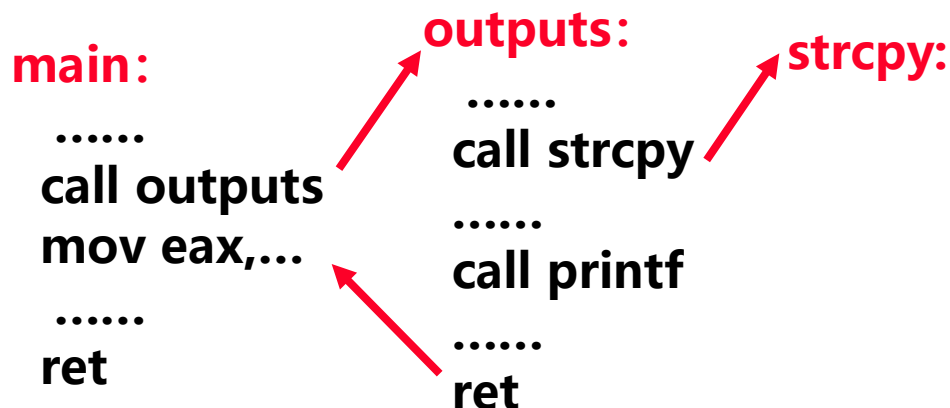
程序及指令的执行过程

- 程序和指令的关系
 - 程序由一条一条指令组成，指令按顺序存放在连续存储单元
- 程序的执行：周而复始地执行一条一条指令
 - 正常情况下，指令按其存放顺序执行
 - 遇到需改变程序执行流程时，用相应的转移指令（包括无条件转移指令、条件转移指令、调用指令和返回指令等）来改变程序执行流程
- 程序的执行流的控制
 - 将要执行的指令所在存储单元的地址由程序计数器PC给出，通过改变PC的值来控制执行顺序
- 指令周期：CPU取出并执行一条指令的时间

程序及指令的执行过程

程序执行流:

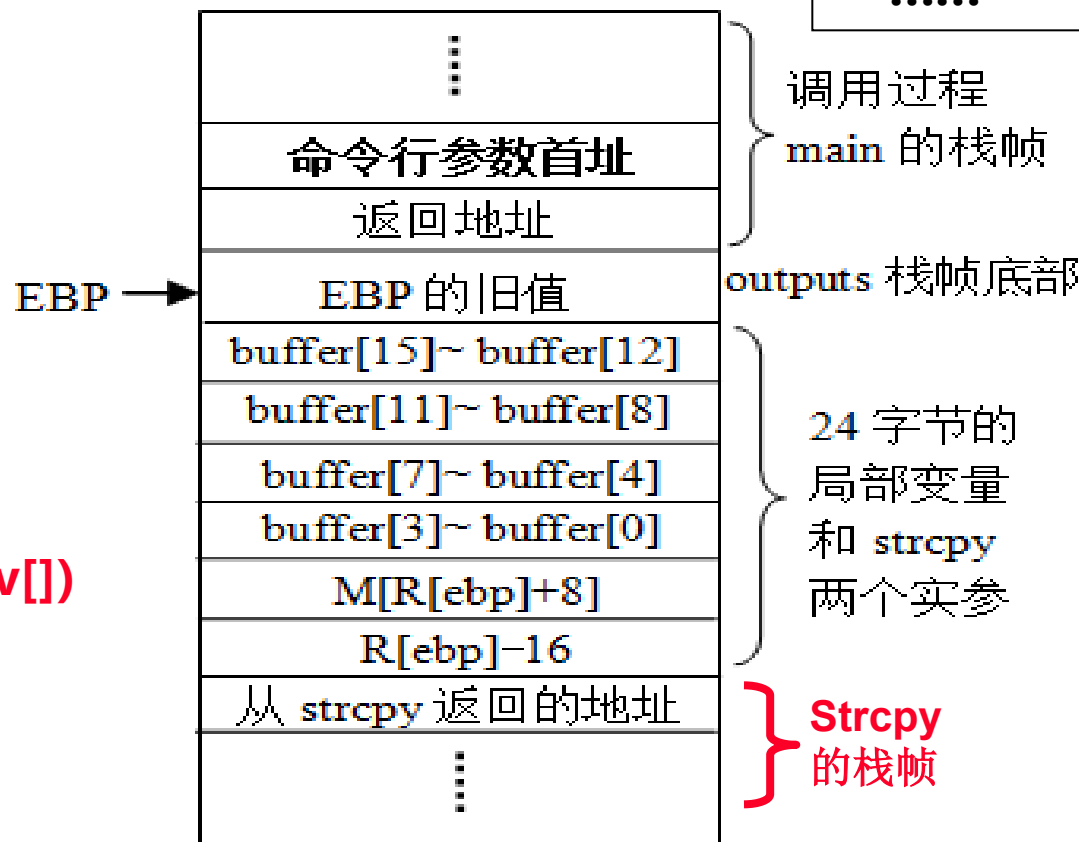
```
.....  
call outputs  
.....  
call strcpy  
.....  
call printf  
.....  
ret  
mov %eax,...  
.....
```



对于3.6.1中的例子

```
#include "stdio.h"  
#include "string.h"  
void outputs(char *str)  
{  
    char buffer[16];  
    strcpy(buffer,str);  
    printf("%s \n", buffer);  
}  
.....
```

```
int main(int argc, char *argv[])  
{  
    outputs(argv[1]);  
    return 0;  
}
```



程序及指令的执行过程

反汇编得到的outputs汇编代码

```
080483e4 : push    %ebp
080483e5 : mov     %esp,%ebp
080483e7 : sub     $0x18,%esp
080483ea : mov     0x8(%ebp),%eax
080483ed: mov     %eax,0x4(%esp)
080483f1 : lea     0xffffffff0(%ebp),%eax
080483f4 : mov     %eax,(%esp)
080483f7 : call    0x8048330 <__gmon_start__@plt+16>
080483fc : lea     0xffffffff0(%ebp),%eax
080483ff : mov     %eax,0x4(%esp)
08048403: movl    $0x8048500,(%esp)
0804840a: call    0x8048310
0804840f : leave
08048410: ret
```

将strcpy的两个实参入栈

将printf的两个实参入栈

程序及指令的执行过程

在内存存放的指令实际上是机器代码（0/1序列）

08048394 <add>:

```
1 8048394: 55          push %ebp
2 8048395: 89 e5       mov %esp, %ebp
3 8048397: 8b 45 0c    mov 0xc(%ebp), %eax
4 804839a: 03 45 08    add 0x8(%ebp), %eax
5 804839d: 5d          pop %ebp
6 804839e: c3          ret
```

° 对于add函数

- ✓ 指令按顺序存放在0x08048394开始的存储空间。
- ✓ 各指令长度可能不同，如push、pop和ret指令各占一个字节，第2行mov指令占两个字节，第3行mov指令和第4行add指令各占3字节。
- ✓ 各指令对应的0/1序列含义有不同的规定，如“push %ebp”指令为01010101B，其中01010为push指令操作码，101为EBP的编号，“pop %ebp”为01011101B，其中01011为pop指令的操作码。

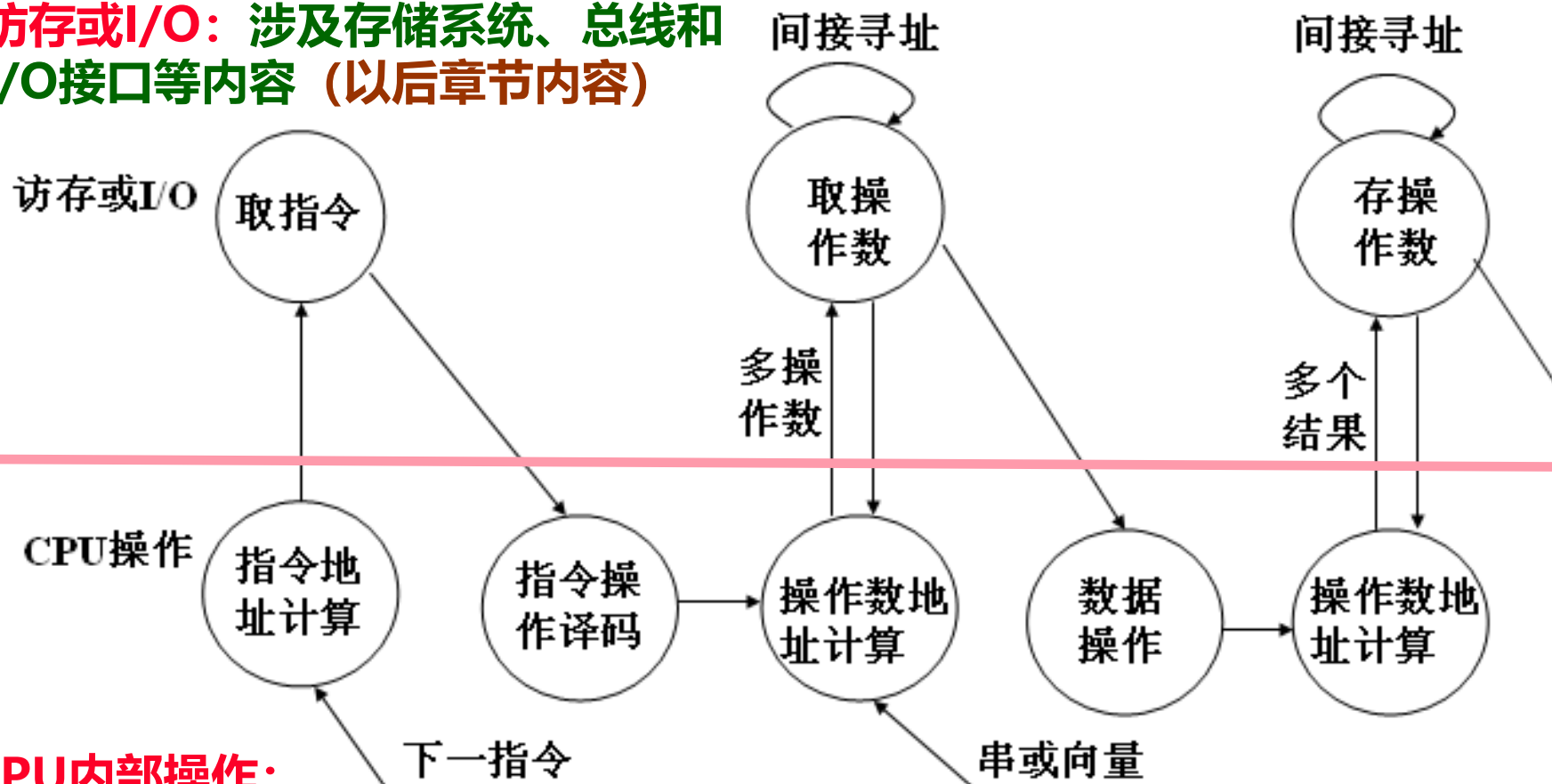
程序执行需要解决的问题：

如何判定每条指令有多长？

如何判定操作类型、寄存器编号、立即数等？如何区分第2行和第3行mov指令的不同？如何确定操作数是在寄存器中还是在存储器中？一条指令执行结束后如何正确读取到下一条指令？

程序及指令的执行过程

访存或I/O：涉及存储系统、总线和I/O接口等内容（以后章节内容）



CPU内部操作：涉及CPU内部数据通路（本章节内容）

CPU运行程序的过程就是执行一条一条指令的过程

CPU执行指令的过程中，包含CPU操作、访问内存或I/O端口的操作两类

机器指令的执行过程

° CPU执行指令的过程

- 取指令
- PC+ “1”
- 指令译码
- 进行主存地址运算
- 取操作数
- 进行算术 / 逻辑运算
- 存结果
- 以上每步都需检测 “异常”
- 若有异常，则自动切换到异常处理程序
- 检测是否有 “中断” 请求，有则转中断处理

取指
阶段

“1”：指一条指令的长度，定长指令字每次都一样；变长指令字每次可能不同

执行
阶段

指令
执行
过程

定长指令字通常在译码前做，变长指令字在译码后做！

问题：

“取指令”一定在最开始做吗？PC+ “1”一定在译码之前做吗？

“译码”须在指令执行前做吗？

你能说出几种 “异常” 事件？“异常” 和 “中断” 的差别是什么？

异常是在CPU内部发生的，中断是由外部事件引起的

机器指令的执行过程

- **取指令：**从PC所指单元取出指令送指令寄存器（IR），并增量PC。
 - 如add函数，开始PC（IA-32的EIP）中存放的是0x0848394，CPU根据PC取指令送IR，每次总是取最长指令字节数，假定最长指令是4个字节，即IR为32位，此时，也即55 89 E5 8BH被取到IR中。
- **指令译码：**不同指令其功能不同，因而需要不同的操作控制信号。
 - CPU根据不同操作码译出不同控制信号。对于上述取到IR中的55 89 E5 8BH译码时，可根据高5位01010译码得到push指令的控制信号。
- **源操作数地址计算并取操作数：**根据寻址方式确定源操作数地址计算方式，若是存储器数据，则需一次或多次访存；若为间接寻址或两操作数都在存储器的双目运算，则需多次访存；若是寄存器数据，则直接从寄存器取数。
- **执行数据操作：**在ALU或加法器等运算部件中对取出的源操作数进行运算。
- **目的操作数地址计算并存结果：**根据寻址方式确定目的操作数地址计算方式，若是存储器数据，则需要一次或多次访存（间接寻址时）；若是寄存器数据，则在进行数据操作时直接存结果到寄存器。
- **指令地址计算并将其送PC。**顺序执行时，PC加上当前指令长度；遇到转移类指令时，则需要根据条件码、操作码和寻址方式等确定下条指令地址。

机器指令的执行过程

- 每条指令的功能总是由以下四种基本操作来实现：

读取某一主存单元的内容，并将其装入某个寄存器（取指，取数）

把一个数据从某个寄存器存入给定的主存单元中（存结果）

把一个数据从某寄存器送到另一寄存器或者ALU（取数，存结果）

进行算术或逻辑运算（ $PC + 1$ ，计算地址，运算）

指令执行过程中查询各种异常情况，并在发现异常时转异常处理

指令执行结束时查询中断请求，并在发现中断请求时响应中断

- 操作功能可形式化描述

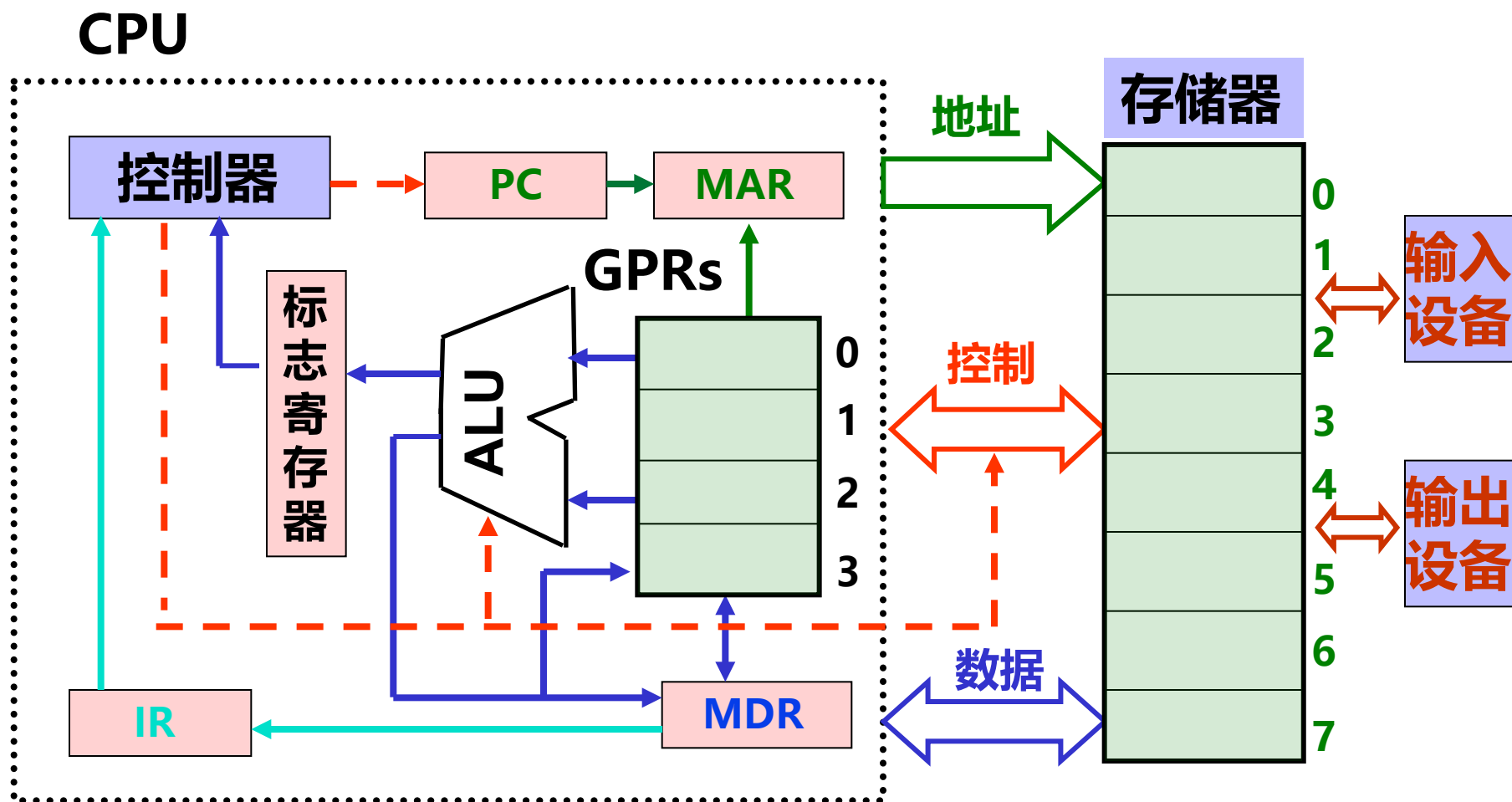
描述语言称为寄存器传送语言RTL (Register Transfer Language)

回顾：冯.诺依曼结构模型机

妈妈会做的菜和厨师会做的菜不一样，同一个菜谱的做法也可能不同

如同

不同架构支持的指令集不同，同一种指令的实现方式和功能也可能不同



回顾： IA-32的寄存器组织

编号	8 位寄存器	16 位寄存器	32 位寄存器	64 位寄存器	128 位寄存器
000	AL	AX	EAX	MM0 / ST(0)	XMM0
001	CL	CX	ECX	MM1 / ST(1)	XMM1
010	DL	DX	EDX	MM2 / ST(2)	XMM2
011	BL	BX	EBX	MM3 / ST(3)	XMM3
100	AH	SP	ESP	MM4 / ST(4)	XMM4
101	CH	BP	EBP	MM5 / ST(5)	XMM5
110	DH	SI	ESI	MM6 / ST(6)	XMM6
111	BH	DI	EDI	MM7 / ST(7)	XMM7

反映了体系结构发展的轨迹，字长不断扩充，指令保持兼容

ST (0) ~ ST (7) 是80位，MM0 ~MM7使用其低64位

程序的执行机制

◦ 分以下三个部分介绍

• 第一讲：程序执行概述

- 程序及指令的执行过程
- CPU的基本功能和基本组成

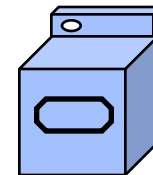
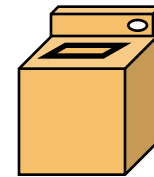
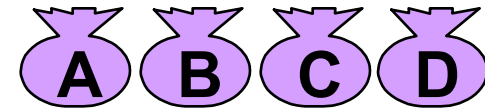
• 第二讲：流水线方式下指令的执行

- 指令流水线的基本原理
- 适合流水线的指令集特征
- CISC和RISC风格指令集
- 指令流水线的实现
- 高级流水线实现技术

一个日常生活中的例子—洗衣服

◦ Laundry Example

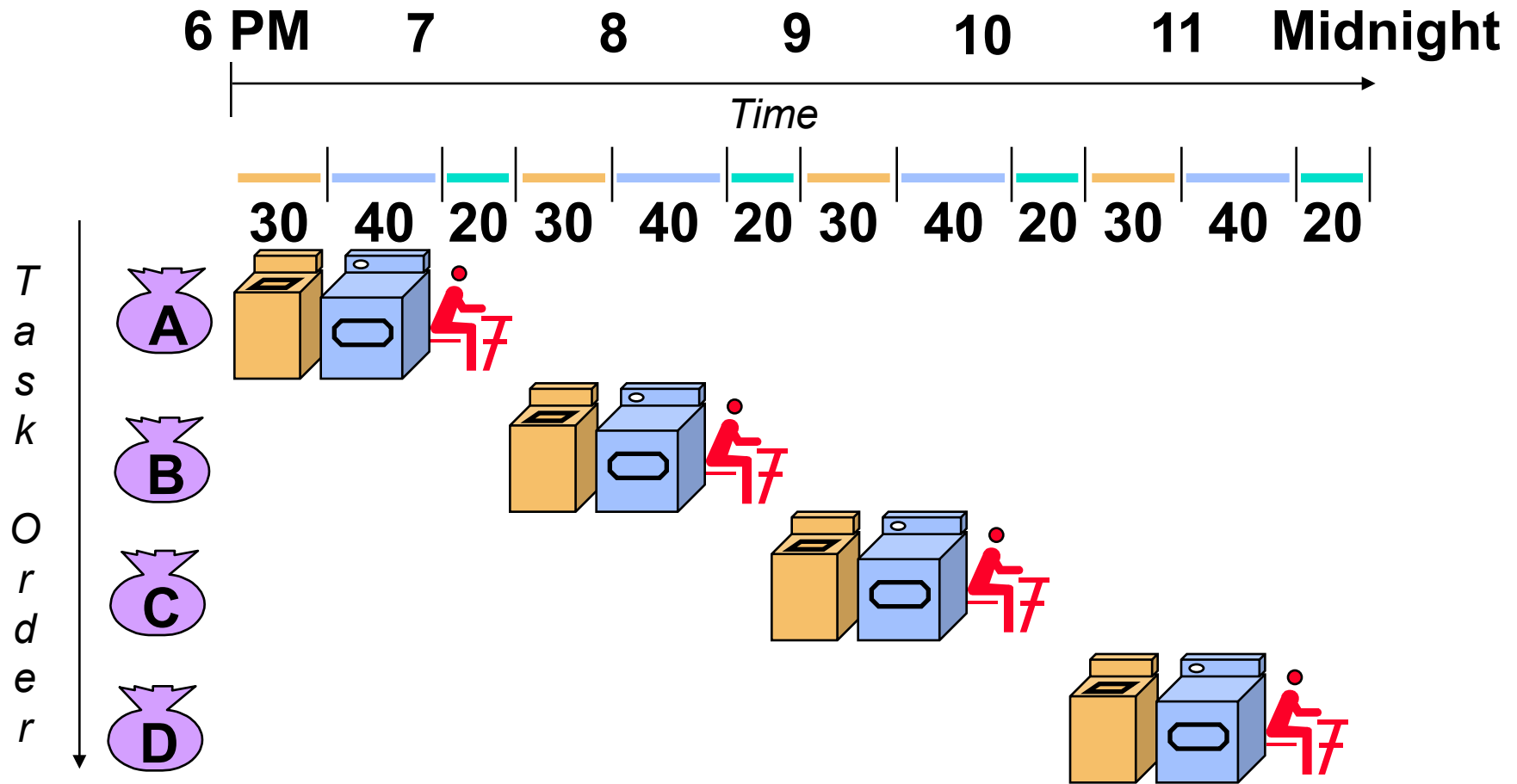
- Ann, Brian, Cathy, Dave each have one load of clothes to **wash, dry, and fold**
- Washer takes **30 minutes**
- Dryer takes **40 minutes**
- “Folder” takes **20 minutes**



如果让你来管理洗衣店，你会如何安排？

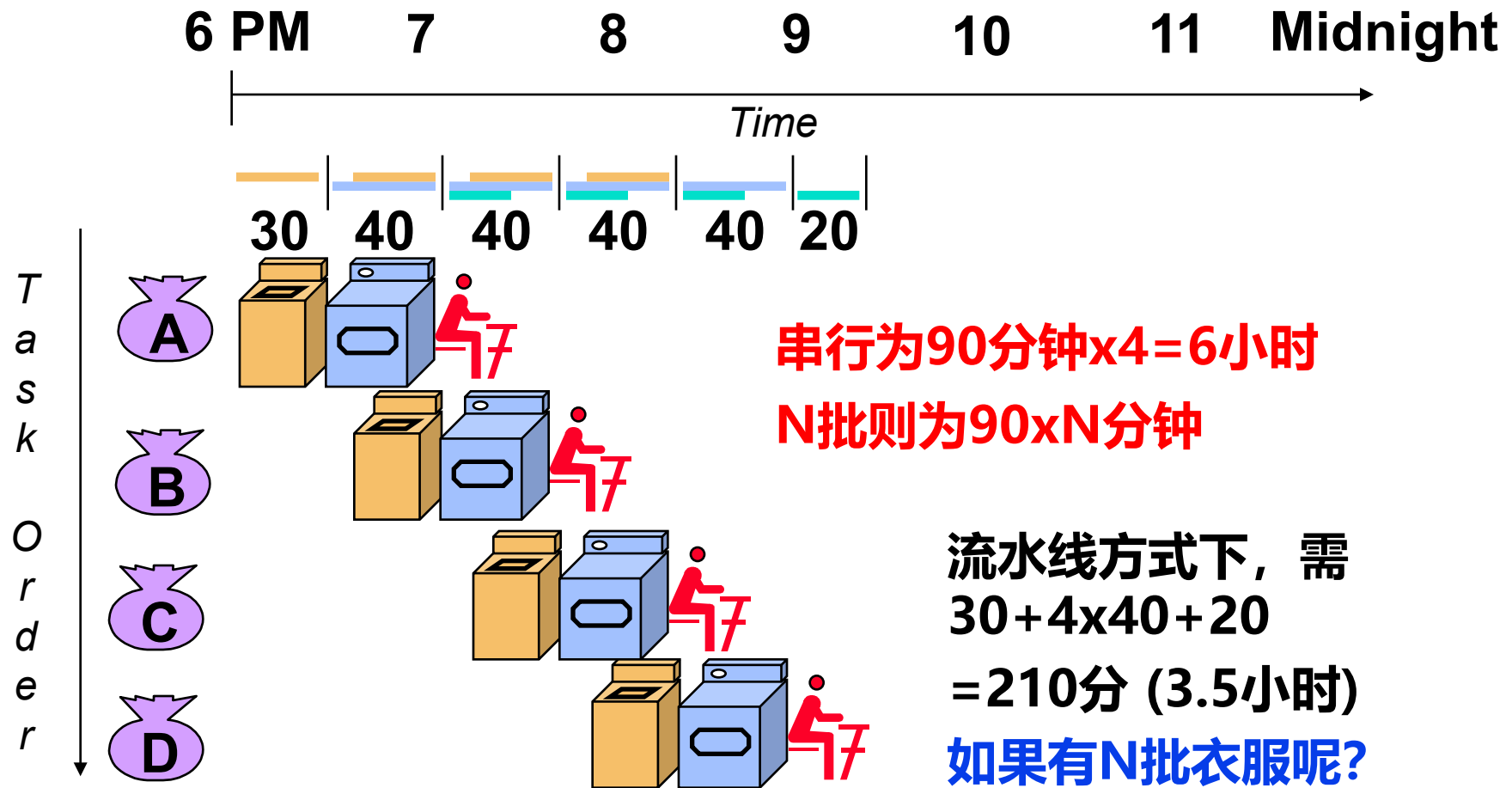
Pipelining: It's Natural !

Sequential Laundry (串行方式)



- 串行方式下 4 批衣服需花费 6 小时 ($4 \times (30 + 40 + 20) = 360$ 分钟)
- N 批衣服, 需花费的时间为 $N \times (30 + 40 + 20) = 90N$
- 如果用流水线方式洗衣服, 则花多少时间呢?

Pipelined Laundry: (Start work ASAP)



流水方式下，所用时间主要与最长阶段的时间有关！

假定每一步时间均衡，则比串行方式提高约3倍！

指令流水线的基本概念

◦ 五段流水线

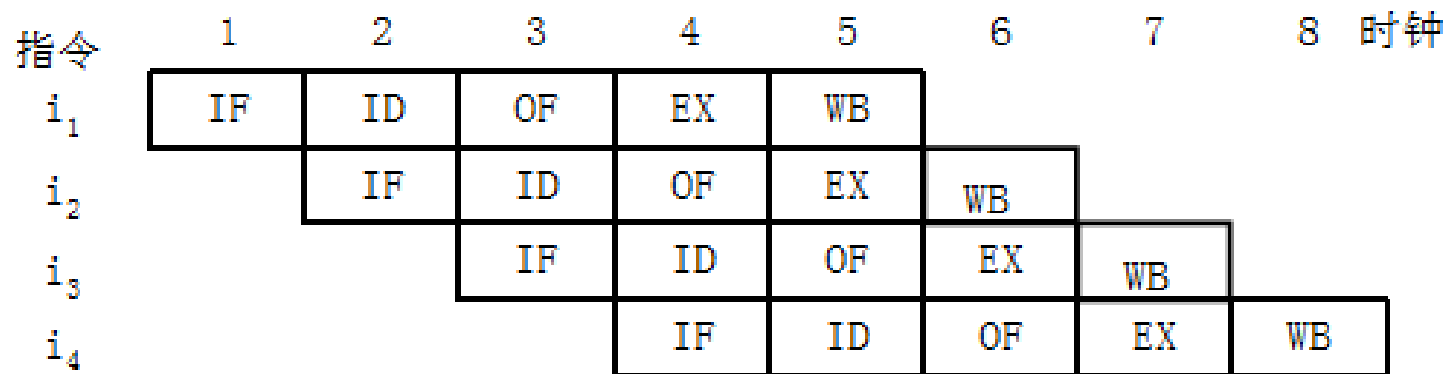
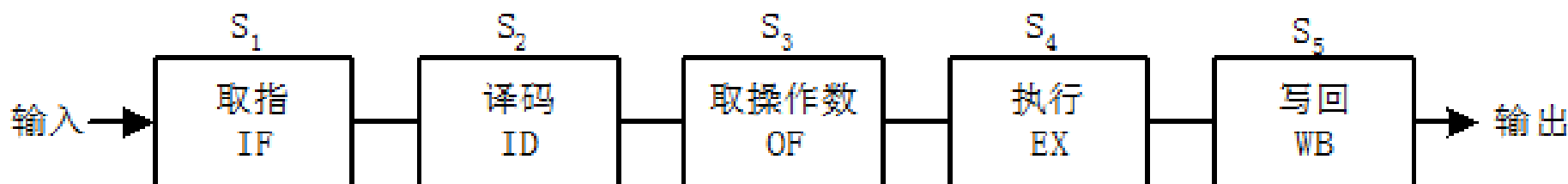
取指令(IF): 根据PC的值从存储器取出指令。

指令译码(ID): 产生指令执行所需的控制信号。

取操作数(OF): 读取存储器操作数或寄存器操作数。

执行(EX): 对操作数完成指定操作。

写回(WB): 将操作结果写入存储器或寄存器。

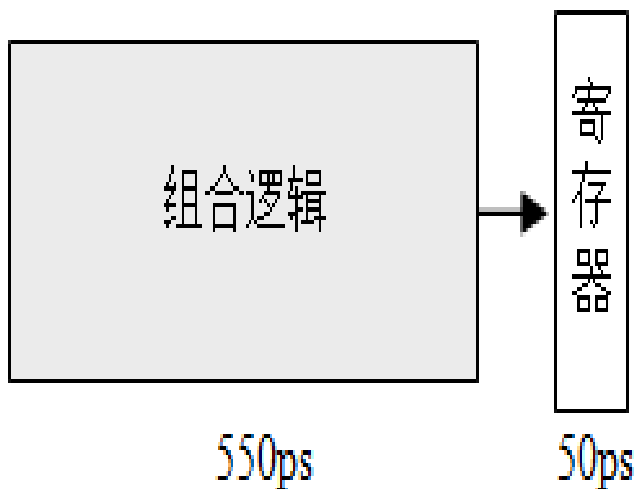


单周期数据通路中指令的执行

假定：最复杂指令执行过程 ① 取指：200ps；②译码和读操作数：50ps；
③ALU操作：100ps；④读存储器：200ps；⑤结果写寄存器：50ps。

$$200 + 50 + 100 + 200 = 550$$

单周期：每条指令在单个时钟周期内完成，
故CPI=1，时钟周期=600ps



CPI=1，指令延时为600ps
指令吞吐率为1.67GIPS

每秒执行指令条数：

$$1/600\text{ps} = 1/(600 \times 10^{-15}) = 1.67 \times 10^{12}$$

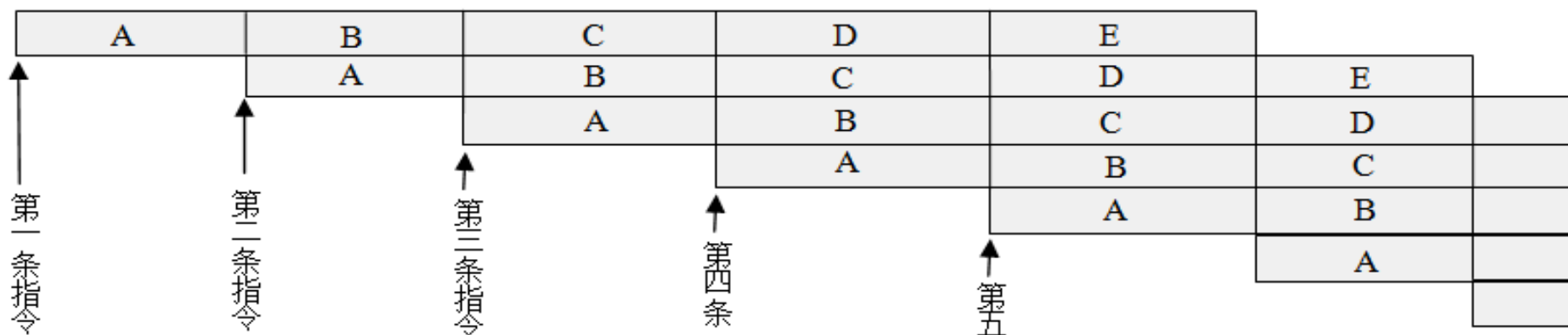
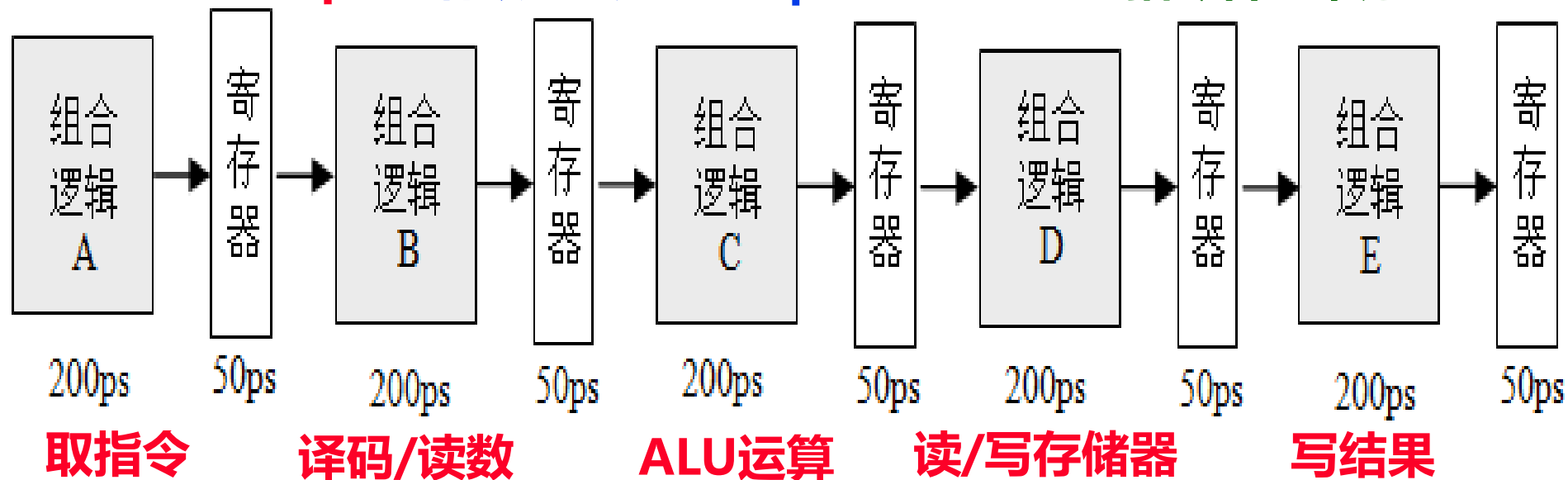
指令串行执行，程序执行时间
为：指令条数×600ps



流水线数据通路中指令的执行

假定：最复杂指令执行过程 ① 取指：200ps；②译码和读操作数：50ps；
③ALU操作：100ps；④读存储器：200ps；⑤结果写寄存器：50ps。

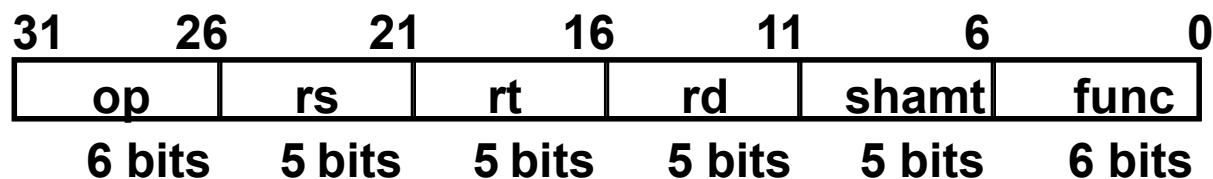
最长段为200ps **指令延时为： $250\text{ps} \times 5 = 1.25\text{ns}$** **指令吞吐率为4GIPS**



流水线指令集的设计

° 具有什么特征的指令集有利于流水线执行呢？

- 长度尽量一致，有利于简化取指令和指令译码操作
 - MIPS指令32位，下址计算方便: $PC+4$
 - X86指令从1字节到17字节不等，使取指部件极其复杂
- 格式少，且源寄存器位置相同，有利于在指令未知时就可取操作数
 - MIPS指令的rs和rt位置一定，在指令译码时就可读rs和rt的值



若位置随指令不同而不同，则需先确定指令类型才能取寄存器编号

- load / Store指令才能访问存储器，有利于减少操作步骤，规整流水线
 - lw/sw指令的地址计算和运算指令的执行步骤规整在同一个周期
 - X86运算类指令操作数可为内存数据，需计算地址、访存、执行
- 内存中“对齐”存放，有利于减少访存次数和流水线的规整

总之，规整、简单和一致等特性有利于指令的流水线执行

按指令格式的复杂度来分

按指令格式的复杂度来分，有两种类型计算机：

复杂指令集计算机CISC (Complex Instruction Set Computer)

精简指令集计算机RISC (Reduce Instruction Set Computer)

早期CISC设计风格的主要特点

- (1) 指令系统复杂
 - 变长操作码 / 变长指令字 / 指令多 / 寻址方式多 / 指令格式多
- (2) 指令周期长
 - 绝大多数指令需要多个时钟周期才能完成
- (3) 各种指令都能访问存储器
 - 除了专门的存储器读写指令外，运算指令也能访问存储器
- (4) 采用微程序控制
- (5) 难以进行编译优化来生成高效目标代码

例如，VAX-11/780小型机

16种寻址方式；9种数据格式；303条指令；一条指令包括1~2个字节的
操作码和下续N个操作数说明符。一个说明符的长度达1~10个字节。

复杂指令集计算机CISC

◆ CISC的缺陷

- 日趋庞大的指令系统不但使计算机的研制周期变长，而且难以保证设计的正确性，难以调试和维护，并且因指令操作复杂而增加机器周期，从而降低了系统性能。

◆ 1975年IBM公司开始研究指令系统的合理性问题，John Cocks提出精简指令系统计算机 **RISC** (Reduce Instruction Set Computer)。

◦ 对CISC进行测试，发现一个事实：

- 在程序中各种指令出现的频率悬殊很大，最常使用的是一些简单指令，这些指令占程序的80%，但只占指令系统的20%。而且在微程序控制的计算机中，占指令总数20%的复杂指令占用了控制存储器容量的80%。

◦ 1982年美国加州伯克利大学的**RISC I**，斯坦福大学的**MIPS**，IBM公司的**IBM801**相继宣告完成，这些机器被称为**第一代RISC机**。

Top 10 80x86 Instructions

° Rank	instruction	Integer Average	Percent total executed
1	load MOV M to R		22%
2	conditional branch Jcc		20%
3	compare CMP		16%
4	store MOV R to M		12%
5	add		8%
6	and		6%
7	sub		5%
8	move register-register		4%
9	call		1%
10	return		1%
	Total		96%

° Simple instructions dominate instruction frequency

(简单指令占主要部分, 使用频率高!)

[BACK](#)

RISC设计风格的主要特点

□ (1) 简化的指令系统

- 指令少 / 寻址方式少 / 指令格式少 / 指令长度一致

□ (2) 以RR方式工作

- 除Load/Store指令可访存外，其余指令都只访问寄存器

□ (3) 指令周期短

- 以流水线方式工作，因而除Load/Store指令外，其他简单指令都只需一个或一个不到的时钟周期就可完成

- (4) 采用大量通用寄存器，以减少访存次数

- (5) 采用硬连线路控制器，不用或少用微程序控制

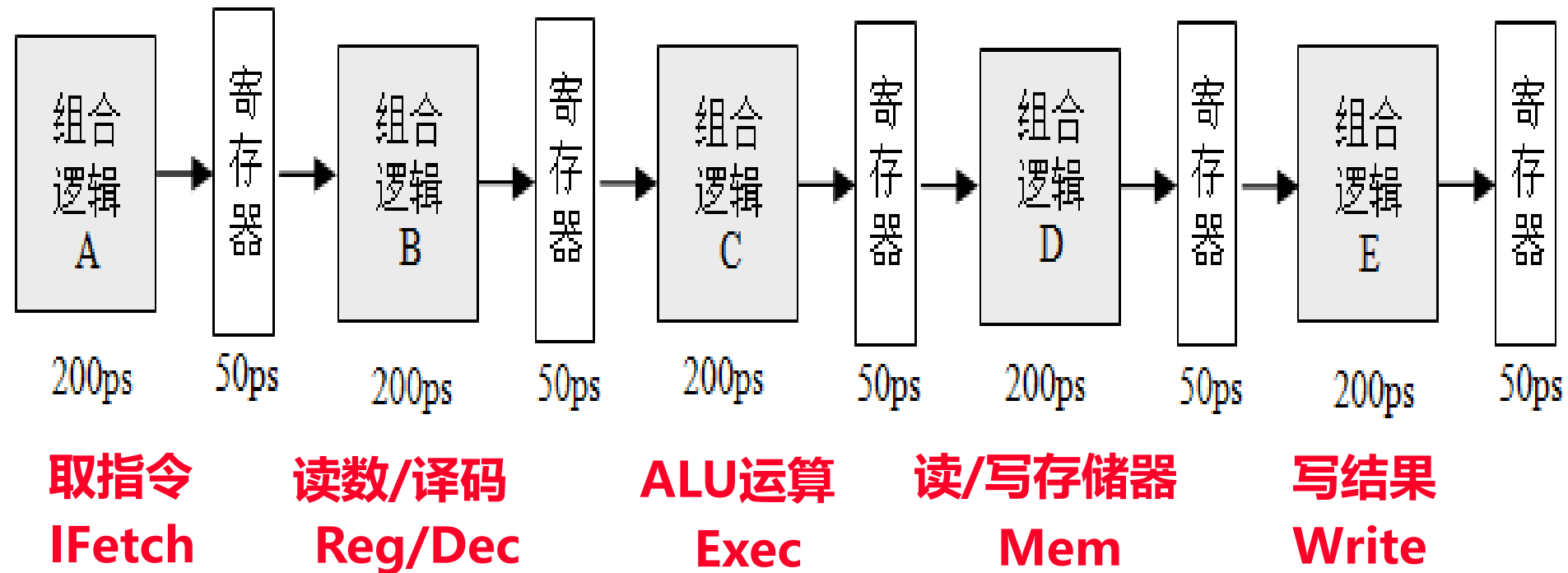
- (6) 采用优化的编译系统，力求有效地支持高级语言程序

MIPS是典型的RISC处理器，82年以来新的指令集大多采用RISC体系结构
x86因为“兼容”的需要，保留了CISC的风格，同时也借鉴了RISC思想

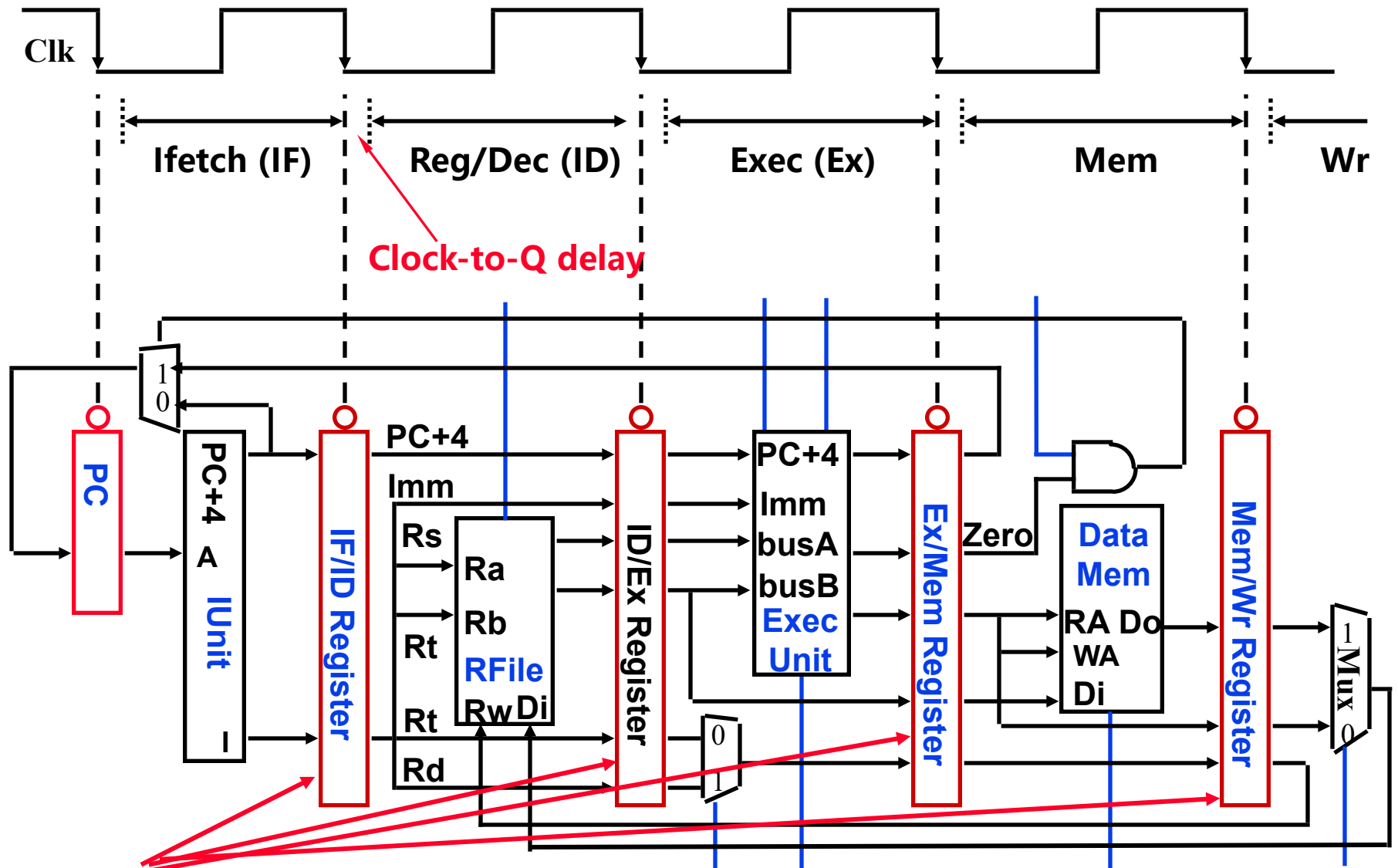
指令流水线的实现

假定：最复杂指令执行过程 ① 取指：200ps；②译码和读操作数：50ps；
③ALU操作：100ps；④读存储器：200ps；⑤结果写寄存器：50ps。

可以分5个流水段，最长阶段为200ps

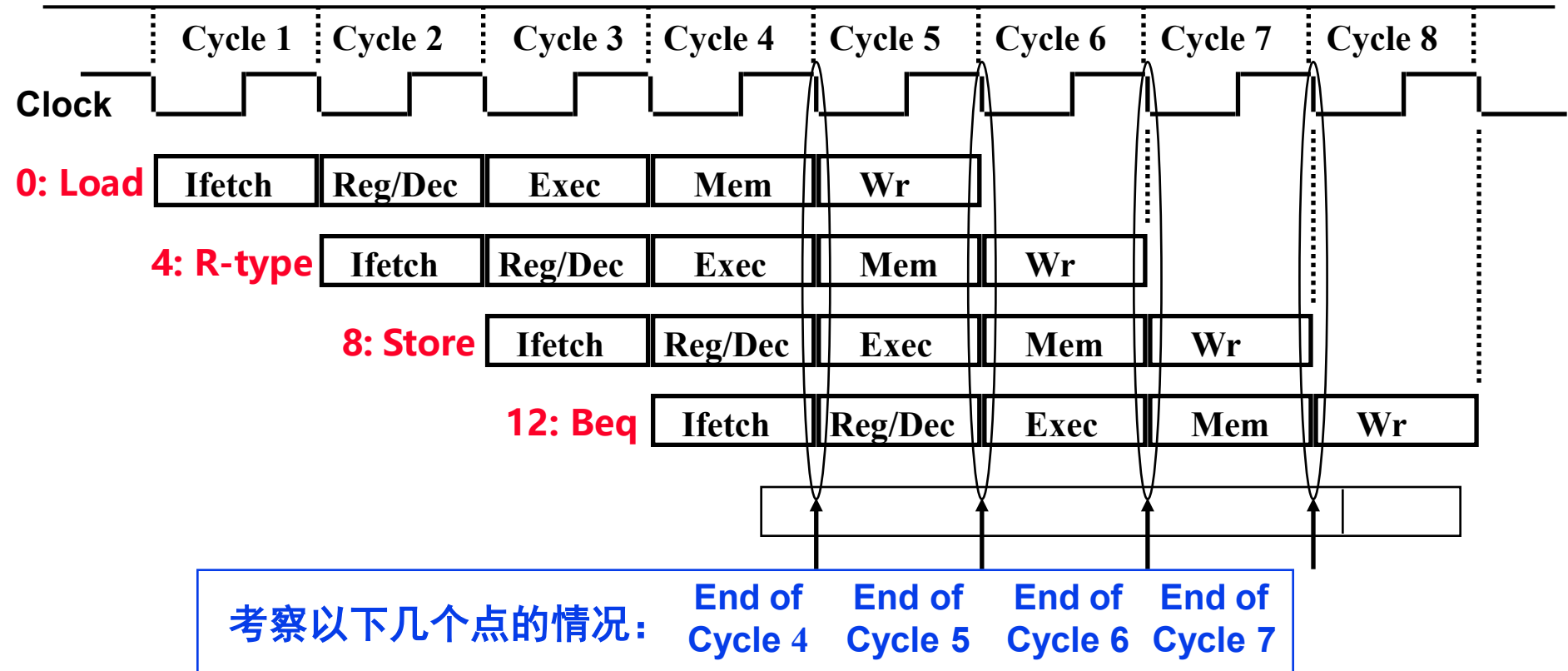


五段流水线数据通路



流水段寄存器：保存每个时钟周期执行的结果！

指令流水线的执行举例



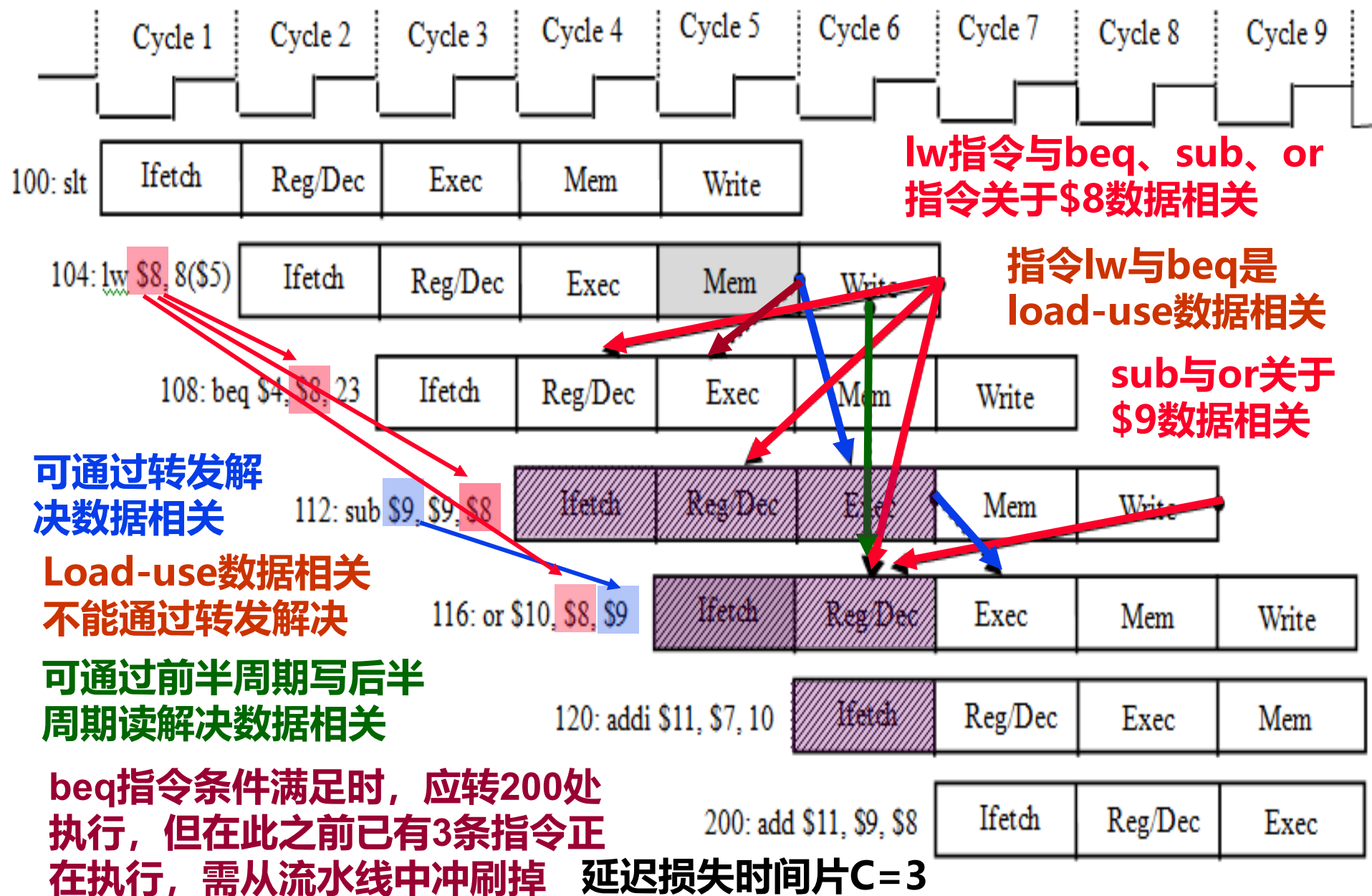
周期4结束: Load 的 Mem, R-type 的 Exec, Store 的 Reg, Beq 的 Ifetch

周期5结束: Load 的 Wr, R-type 的 Mem, Store 的 Exec, Beq 的 Reg

周期6结束: R-type 的 Wr, Store 的 Mem, Beq 的 Exec

周期7结束: Store 的 Wr, Beq 的 Mem

指令流水线的执行举例



流水线的冲突/冒险(hazard)情况

○ Hazards: 指流水线遇到无法正确执行后续指令或执行了不该执行的指令

- 结构冒险 (hardware resource conflicts, 硬件资源冲突):

现象: 同一个部件同时被不同指令所使用

- 一个部件每条指令只能使用1次, 且只能在特定周期使用
- 设置多个部件, 以避免冲突。如指令存储器IM 和数据存储器DM分开

- 数据冒险 (data dependencies, 数据相关):

现象: 后面指令用到前面指令结果数据时, 前面指令的结果还没产生

- 转发(Forwarding/Bypassing旁路) 或 前半周期读后半周期写
- Load-use冒险不能通过转发解决, 需阻塞(stall)一个时钟周期
- 编译程序优化指令顺序

- 控制 (分支) 冒险 (changes in program flow, 改变控制流):

现象: 转移或异常改变执行流程, 后继指令在目标地址产生前已被取出

- 采用静态或动态分支预测
- 编译程序优化指令顺序(分支延迟)

本PPT内容只需大概了解

编译器优化指令顺序解决数据冒险

以下源程序可生成两种不同的代码，优化的代码可避免Load阻塞

$a = b + c;$

$d = e - f;$

假定 a, b, c, d, e, f 在内存

编译器的优化很重要!

Slow code:

```
lw    $2, b
lw    $3, c
add   $1, $2, $3
sw    $1, a
lw    $5, e
lw    $6, f
sub   $4, $5, $6
sw    $4, d
```

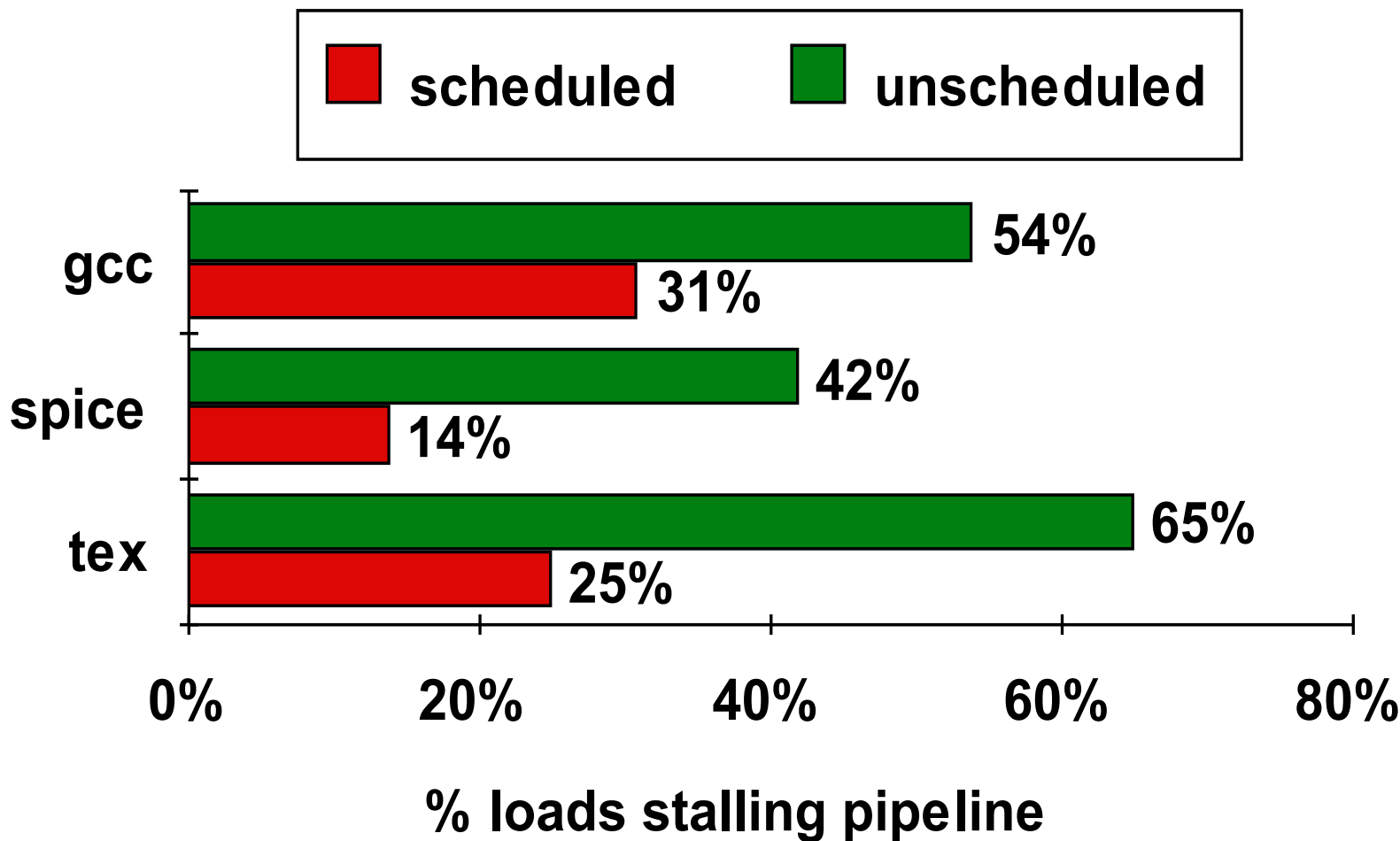
调整后

Fast code:

```
lw    $2, b
lw    $3, c
lw    $5, e
add   $1, $2, $3
lw    $6, f
sw    $1, a
sub   $4, $5, $6
sw    $4, d
```

如果硬件不支持阻塞处理的话，则编译器可以将顺序调整和插入NOP指令结合起来，在找不到可插入的指令时，插入NOP指令!

编译器优化以避免阻塞的情况调查



由此可见，优化调度后load阻塞现象大约降低了1/2~1/3

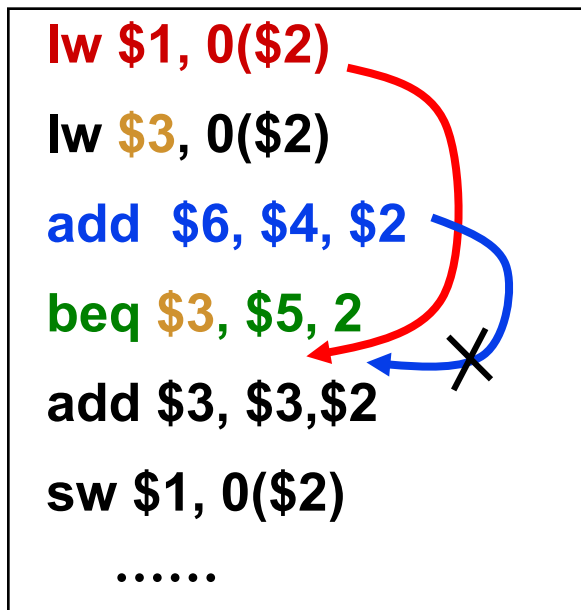
编译器优化指令顺序解决控制冒险

- 基本思想：把分支指令前面的与分支指令无关的指令调到分支指令后面执行，以填充延迟时间片（也称分支延迟槽Branch Delay slot），不够时用nop填充

如何对以下程序段进行分支延迟调度？（假定时间片为2）

若分支延迟时间片减少为1

```
lw $1, 0($2)
lw $3, 0($2)
add $6, $4, $2
beq $3, $5, 2
add $3, $3, $2
sw $1, 0($2)
.....
```



调度后

```
lw $3, 0($2)
add $6, $4, $2
beq $3, $5, 2
lw $1, 0($2)
nop
add $3, $3, $2
sw $1, 0($2)
```

```
lw $3, 0($2)
add $6, $4, $2
beq $3, $5, 2
lw $1, 0($2)
add $3, $3, $2
sw $1, 0($2)
.....
```

调度后可能带来其他问题：
产生新的load-use数据冒险

调度后，无需在硬件线路中阻塞
branch指令后面指令的执行

提高性能措施—实现指令级并行

- 实现指令流内部的并行流水线称为指令级并行 (ILP)
- 有两种指令级并行策略
 - 超流水线 (Super- pipelining)
 - 级数更多的流水线 $CPI = ?$ $CPI = 1$
 - 理想情况下, 流水线的加速比与流水段的数目成正比
(即: 理想情况下, 流水段越多, 时钟周期越短, 指令吞吐率越高)
但是, 它会增加开销, 且是有极限的! 可以怎样突破极限呢?
 - 多发射流水线 (Multiple issue pipelining)
 - 多条指令(如整数、浮点、装入/存储等) 同时启动并独立运行
 - 前提: 有多个执行部件。如定点、浮点、乘/除、取数/存数部件等
 - 结果: 能达到小于1的CPI, 定义CPI的倒数为IPC
(例如: 理想的四路多发射流水线的IPC为4)
 - 两种实现方法
 - 静态多发射: 由编译器在编译时静态完成指令打包和冒险处理
 - 动态多发射: 由硬件在执行时动态完成指令打包和冒险处理

N段流水线说明一个时钟周期内最多有几条指令同时并行执行?

N条! 故N越大并行度越高!

静态多发射处理器

- 由编译器在编译时进行相关性分析和静态分支预测，以静态完成指令打包
 - 指令打包（将同时发射的多条指令合并到一个长指令中）
 - 将同一个时钟周期内发射的多个指令看成是一条多个操作的长指令，称为一个“发射包”
 - “静态多发射”也被称为“超长指令字”（VLIW-Very Long Instruction Word），采用这种技术的处理器被称为VLIW处理器
 - 在同一个周期内发射的指令类型是受限制的（举例:干洗/水洗）
例如，只能是一条ALU指令/分支指令、一条Load/Store指令
 - IA-64采用这种方法，Intel称其为EPIC（Explicitly Parallel Instruction Computer—显式并行指令计算机） 安腾、安腾2

静态打包时，一定要保证指令内部不会出现冒险！

动态多发射处理器

- 由硬件在执行时动态完成指令打包或冒险处理
- 通常被称为超标量处理器 (Superscalar)
 - 同一个时钟动态发射多条指令，一个周期内可执行一条以上指令
- 与VLIW处理器的不同点：
 - **VLIW处理器**：与机器结构密切相关，在结构有差异机器上要重新编译
 - **超标量处理器**：编译器仅进行指令顺序调整（还是串行序列），不进行指令打包，而是由硬件根据机器结构决定同时发射哪几条指令。因此，编译后的代码能够被不同结构的机器正确执行
- 超标量多结合动态流水线调度 (Dynamic pipeline scheduling) 技术
 - 通过指令相关性检测和动态分支预测等手段，投机性地不按指令顺序执行，当发生流水线阻塞时，可以到后面找指令来执行（乱序执行）
 - 举例说明动态流水线调度技术：

```
lw      $t0, 20($s2)
addu    $t1, $t0, $t2
sub      $s4, $s4, $t3
slti    $t5, $s4, 20
```

左边指令序列中，哪条指令可以提前执行？

sub指令可提前到addu指令前执行

如果不将sub调到前面，则会影响slti指令的执行，而且还会发生load-use冒险

本章小结

- CPU的基本功能是周而复始地执行指令，并处理异常和中断。
- CPU最基本的部分是数据通路和控制单元
 - 数据通路 (datapath) 中包含组合逻辑元件和存储信息的状态元件。
 - 组合逻辑 (如加法器、ALU、扩展器、多路选择器以及状态元件的读操作逻辑等) 用于对数据进行处理;
 - 状态元件包括触发器、寄存器和存储器等, 用于对指令执行的中间状态或最终结果进行存储。
 - 控制单元 (control unit) : 对取出的指令进行译码, 与指令执行得到的条件标志或当前机器的状态、时序信号等组合, 生成对数据通路进行控制的控制信号。
- 指令执行过程主要包括取指、译码、取数、运算、存结果。
- 通常把取出并执行一条指令的时间称为指令周期, 它由机器周期或直接由时钟周期组成。现代计算机已经没有机器周期的概念。

本章小结

- 现代计算机的每个指令周期直接由时钟周期（节拍）组成。
- 时钟信号是CPU中用于控制同步的信号。
- 每条指令功能不同，因此每条指令执行时数据在数据通路中所经过的部件和路径也可能不同。但是，每条指令在取指令阶段都一样。
- 早期计算机中数据通路采用总线方式，通过CPU的内部总线把CPU中的通用寄存器、ALU、暂存器、指令寄存器等互连，有单总线、二总线和三总线结构数据通路。
- 现代计算机的数据通路都采用流水线方式实现，将每条指令的执行过程分解成功能段相同的几个流水段，每个流水段的执行时间也被设置成完全相同。
- 流水线方式下，同时有多条指令重叠执行，因此程序的执行时间比串行执行方式下缩短很多。在有些情况下会发生流水线冒险，包括结构冒险、数据冒险和控制冒险三类。
- 高级流水线：超流水、静态多发射（VLIW）、超标量、动态调度
- 编译器优化：静态调度、循环展开、...